GRAPH THEORY PROJECT

Water Jug Problem Solver

Finding a minimum-step solution with Breadth-First Search

A group project by

Glenda Pinhero · Gouri Krishna A

Gayathri Jayan · Abhirami Mohan

01 / Understanding the problem

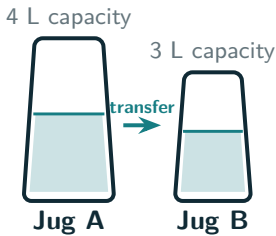
THE CHALLENGE

Can two fixed-capacity jugs measure an exact amount?

The task is to obtain a target quantity of water using two jugs, without measurement markings.

- Each jug has a fixed capacity.
- Water can be added, discarded, or transferred.
- The goal is to reach a state that contains the target amount.

Why it matters: a simple physical puzzle introduces search, planning, and algorithmic problem-solving.



Example goal

Measure 2 litres using 4 L and 3 L jugs.

02 / Permitted operations

PERMITTED OPERATIONS

1. **Fill** a jug to its capacity.
2. **Empty** a jug completely.
3. **Pour** from one jug into the other until the source is empty or the destination is full.

Each operation changes the configuration, but never exceeds a jug's capacity.

03 / Model it as a graph

STATE-SPACE MODEL

- Each node represents one valid state (a, b).
- Each directed edge represents one permitted operation.
- The initial node is $(0, 0)$.
- A goal node contains the target quantity.

This turns the puzzle into a search problem: explore valid transitions until a goal state is found.



Example states for 4 L and 3 L jugs

The complete graph depends on the selected jug capacities.

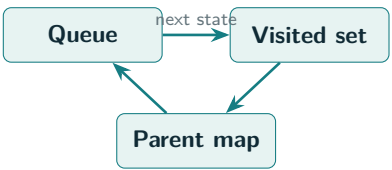
04 / Why Breadth-First Search?

THE SEARCH STRATEGY

BFS explores states in layers: first the start state, then all states one operation away, then two operations away, and so on.

1. Add the start state to a FIFO queue.
2. Remove the next state from the front.
3. Generate valid next states.
4. Record unseen states and continue until a goal is found.

KEY DATA STRUCTURES



Why BFS?

If every operation has equal cost, BFS finds a solution with the fewest operations.

05 / Implementation: valid jug operations

CORE LOGIC

The solver applies the permitted operations to a state and produces its valid neighbours.

```
def next_states(a, b, cap_a, cap_b):
    # Fill either jug
    yield (cap_a, b)
    yield (a, cap_b)

    # Empty either jug
    yield (0, b)
    yield (a, 0)

    # Pour A -> B
    move = min(a, cap_b - b)
    yield (a - move, b + move)

    # Pour B -> A
    move = min(b, cap_a - a)
    yield (a + move, b - move)
```

Illustrative core logic: in a complete implementation, repeated states are filtered using a visited set. Function names can be aligned with the final project source.

06 / Implementation: BFS solver

SEARCH + PATH TRACKING

A queue drives the search; a parent map stores how each state was reached so the path can be reconstructed.

```
from collections import deque

def bfs(start, is_goal, neighbours):
    queue = deque([start])
    parent = {start: None}

    while queue:
        state = queue.popleft()
        if is_goal(state):
            path = []
            while state is not None:
                path.append(state)
                state = parent[state]
            return path[::-1]

        for nxt in neighbours(state):
            if nxt not in parent:       # unseen state
                parent[nxt] = state
                queue.append(nxt)

    return None  # no solution found
```

TRY THE PROJECT

Explore the Water Jug Solver

Watch the states change as BFS finds a solution.

LIVE DEMO WEBSITE

water-jug-solver.streamlit.app

https://water-jug-solver-zgcfjkasx5rqv2nsggwbnf.streamlit.app/